

CloudShield AI / DAIL

08 — Impact & Invalidation Specification

Version 1.0 | Final Impact/Invalidation Algorithm Baseline | 1 September 2026

Purpose: Define the deterministic algorithm by which DAIL converts resource changes and dependency relationships into a conservative set of protected invariants whose previous evidence can no longer be assumed current. This specification defines impact scope, invalidation causes, dependency traversal, uncertainty propagation, evidence carry-forward boundaries, verification work-set generation, and explanations for every affected invariant.

Document Control

Field	Value
Document ID	CSAI-DAIL-IMPACT-008
Status	FINAL ALGORITHM BASELINE
Parent documents	01 SRS; 02 Architecture; 03 Functional; 04 Terraform/AWS; 05 Domain/Data; 06 Lifecycle; 07 Identity/Dependency
Primary inputs	IdentityReport, DependencyGraph, TrustedState, CandidateState, Invariant Registry
Primary output	ImpactReport + Verification Work Set
Initial algorithm	Deterministic graph-based impact analysis
Core principle	Affected does not mean violated; stale evidence must be re-established.

1. Scope

The Impact Engine answers: “Given the candidate transition, which protected obligations may have been invalidated and therefore require current verification?” It does not decide whether an invariant is actually violated and does not authorize promotion.

- Compute changed-resource set.
- Map direct changes to invariant scope.
- Traverse supported dependency graph relationships.
- Propagate identity/dependency uncertainty.
- Determine evidence invalidation.
- Generate required verification work.
- Explain every impact decision.

2. Core Definitions

Term	Definition
Changed resource	Resource classified as CREATED, DELETED, MODIFIED, REPLACED, or UNCERTAIN.
Affected invariant	Protected obligation whose prior evidence may be stale because relevant state context changed.
Direct impact	Candidate change directly touches an invariant’s declared scope.

Dependency impact	Candidate change reaches invariant scope through supported dependency edges.
Invalidation	Loss of eligibility for prior evidence to prove the invariant for the new candidate/trusted state.
Verification work set	Explicit set of properties/invariants requiring current verification.
Impact closure	Reachable impact context under the supported graph and invariant policy.

3. Non-Equivalence Rules

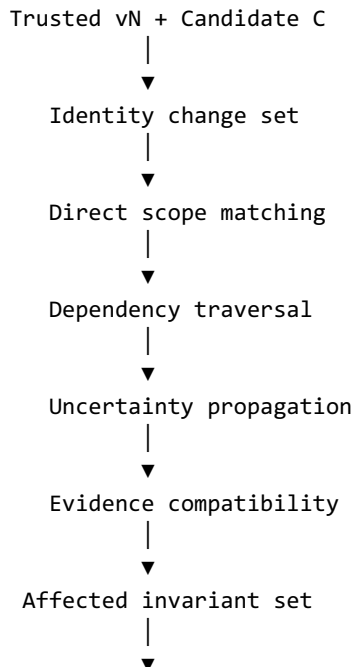
AFFECTED $\neq$ VIOLATED
 AFFECTED $\neq$ UNKNOWN
 NO EDGE $\neq$ NO DEPENDENCY
 NO PRIOR EVIDENCE $\neq$ PASS

Impact is a pre-verification operation. It determines what must be checked; verification determines what is true.

4. Inputs

- Current Trusted State and protected invariant references.
- Candidate State.
- IdentityReport from Document 07.
- DependencyGraph from Document 07.
- Invariant definitions and scopes.
- Evidence compatibility metadata.
- Impact algorithm/policy version.

5. Impact Pipeline



6. Changed Resource Set

Change type	Impact treatment
UNCHANGED	Not an impact source by itself.
MODIFIED	Potential impact source.
CREATED	Potential impact source; baseline checks may apply.
DELETED	Potential impact source; dependent obligations may be affected.
REPLACED	Potential impact source; treat conservatively because identity/runtime context changed.
UNCERTAIN	Potential impact source; propagate uncertainty to relevant obligations.

A resource with an unchanged fingerprint but uncertain identity is not treated as safely unchanged.

7. Invariant Scope Matching

Each invariant declares scope over resources, resource types, relationships, or properties.

```
directly_affected(invariant, changed_resource):
    return changed_resource intersects invariant.scope
```

- Explicit resource addresses/selectors.
- Resource types.
- Security properties/rules.
- Network relationships.
- Source/destination pairs.
- Relevant dependency relationships.
- Property selectors defined by the invariant.

8. Direct Impact Algorithm

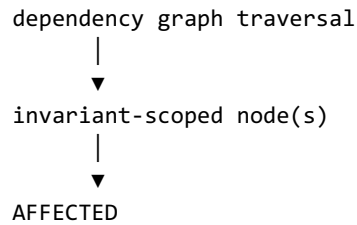
```
for invariant in protected_invariants:
    for change in changed_resources:
        if scope_matches(invariant.scope, change):
            mark AFFECTED
            record DIRECT reason
```

The exact scope element causing the match must be recorded.

9. Dependency-Based Impact

An invariant may be affected even when the changed resource is not directly named in its scope if a supported dependency path connects the change to an invariant-relevant resource or semantic node.

```
changed resource
|
▼
```



Traversal is bounded by supported edge types and invariant-specific impact policy.

10. Traversal Policy

Edge type	Default impact behavior
Terraform reference	Traverse when the referenced object can influence scoped semantics.
Infrastructure relationship	Traverse when defined as impact-relevant for the invariant.
Invariant scope	Terminates traversal at the target obligation.
Invariant semantic relationship	Traverse declared source/destination/context relationships.
UNCERTAIN edge	Propagate uncertainty conservatively.
UNSUPPORTED edge	Cannot prove absence of impact; mark result uncertain where required.

11. Impact Direction

The graph may be stored with semantic dependency direction, but impact analysis must support reverse adjacency. A changed upstream resource may affect a downstream scoped resource.

```
Route Table → Subnet → EC2
change(Route Table)
↓
EC2 network context may change
↓
INV-FUNC-001 may be affected
```

The Impact Engine may therefore maintain reverse indexes without changing the canonical graph semantics.

12. Dependency Closure

```
closure(change):
    frontier = {change}
    visited = {}
    while frontier:
        node = pop(frontier)
        if node in visited:
            continue
        visited.add(node)
        for edge in impact_edges(node):
            if edge is known/supported:
                frontier.add(edge.target)
            else:
```

```
        record uncertainty
    return visited
```

A visited set is mandatory to prevent cycles from causing infinite traversal.

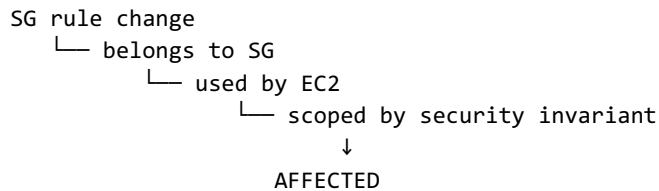
13. Why Typed Impact Matters

Unrestricted graph reachability can over-invalidate unrelated obligations. The MVP therefore uses typed edges plus invariant-specific impact relevance rather than indiscriminate transitive closure.

- Only supported edge types participate.
- Invariant definitions identify relevant relationship classes.
- Traversal reasons are recorded.
- Conservative over-approximation is allowed where policy requires it; silent under-approximation is not.

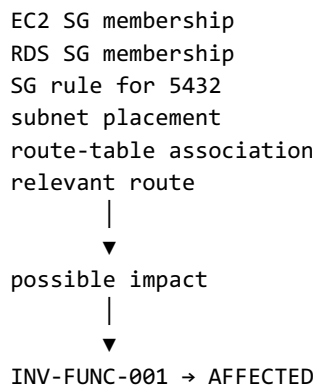
14. Security Invariant Impact

For the canonical public-SSH security invariant, security-group rule changes directly affect the invariant. Changes to which security group applies to an EC2 instance can affect it through supported dependency edges.



15. Functional Invariant Impact

For the canonical EC2–RDS TCP/5432 invariant, relevant security, network placement, routing, source/destination, and supported topology changes may invalidate prior evidence.



The reachability predicate itself is verified later; impact only decides whether old evidence remains sufficient.

16. Created-Resource Baseline Impact

A newly created resource has no historical evidence for the new resource. Absence of a prior invariant does not imply absence of an obligation.

- Identify created resources within supported policy scope.
- Identify applicable baseline predicates.
- Generate baseline verification work.
- Do not infer safety from lack of historical coverage.

17. Deletion Impact

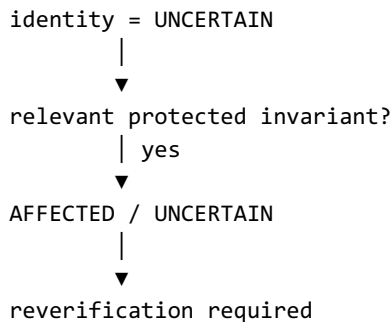
- A deleted resource inside an invariant scope directly affects that invariant.
- Traverse supported reverse dependencies from the deleted node.
- Record deletion as an explicit reason.
- Require current verification for resulting obligations.

18. Replacement Impact

Replacement changes configuration semantics and runtime identity. It is at least as conservative as modification.

- Invalidate evidence tied to the old identity when relevant.
- Traverse dependencies from the replacement context.
- Require current verification for affected obligations.

19. Identity Uncertainty



Ambiguous resource correspondence must never preserve prior proof merely because one possible interpretation appears safe.

20. Dependency Uncertainty

Condition	Required result
Uncertain edge on plausible path to invariant	AFFECTED/UNCERTAIN according to policy.
Unsupported edge prevents proving no impact	Do not claim unaffected; use conservative verification.
Uncertain identity on path	Propagate uncertainty.
Unknown external relationship	Preserve uncertainty and block proof of unaffectedness when required.

21. Evidence Invalidation

```
evidence_valid_for_candidate(e, invariant, candidate):  
    if direct impact: False  
    if dependency impact: False  
    if relevant identity/dependency uncertainty: False  
    if compatibility requirements fail: False  
    otherwise: eligible only if carry-forward policy permits
```

Historical evidence is retained. Its eligibility for a new state/candidate is what becomes invalid.

22. Carry-Forward Eligibility

Condition	Prior evidence
Unaffected + compatible context	May carry forward if explicit policy permits.
Directly affected	Invalid for new state; reverify.
Dependency affected	Invalid for new state; reverify.
Identity uncertain	Not safely reusable.
Dependency uncertain	Not safely reusable where relevant.
Verifier/configuration incompatible	Not reusable.
Invariant definition version changed	Not reusable for the new version.

23. Impact Report Contract

```
ImpactReport {  
    report_id  
    parent_state_id  
    candidate_id  
    changed_resources[]  
    affected_invariants[]  
    verification_work[]  
    uncertainty[]  
    carry_forward_candidates[]  
    reasons[]  
    algorithm_version  
    graph_version  
}
```

24. Impacted Invariant Record

```
ImpactedInvariant {  
    invariant_id  
    invariant_version  
    status = AFFECTED | UNCERTAIN  
    impact_type = DIRECT | DEPENDENCY | IDENTITY | UNCERTAINTY | BASELINE  
    changed_resource_refs[]  
    dependency_paths[]  
    reason  
    prior_evidence_ids[]  
    evidence_validity = INVALID | UNCERTAIN  
}
```

25. Verification Work Set

```
VerificationWorkItem {  
  target_id  
  target_type  
  reason  
  scope  
  required_evidence  
  priority  
}
```

Source	Example work
Security impact	Recheck public SSH predicate.
Functional dependency impact	Recheck EC2–RDS TCP/5432.
Created-resource baseline	Check exposure of new resource.
Identity uncertainty	Re-establish invariant evidence for ambiguous context.
Dependency uncertainty	Use an available verifier to resolve the relevant property.

26. Priority Semantics

Priority	Meaning
BLOCKING	Required before promotion.
HIGH	Strong relevance to a protected obligation.
NORMAL	Required under invariant policy but can be sequenced.
ADVISORY	Non-blocking analysis only when explicitly allowed.

Priority never changes whether a required invariant is mandatory.

27. Impact Explanation Contract

```
changed resource  
  → dependency relationship/path  
  → invariant scope  
  → impact reason  
  → prior evidence invalidated  
  → verification requested
```

Every affected invariant must be explainable using this chain.

28. Canonical Regression Example

```
Trusted v0:  
  SEC = PROTECTED  
  FUNC = PROTECTED
```

```
Candidate R1:  
  security-group rule changed
```

```
Identity:  
  rule = MODIFIED
```

```
Impact:  
  SEC = AFFECTED
```


FUNC = AFFECTED if its declared scope/dependency model includes this path

Verification:

SEC = PASS

FUNC = FAIL

Promotion:

REJECT

The functional impact of a security-group change must be determined by the declared invariant scope and graph, not by an informal assumption.

29. Canonical Safe Repair Example

Candidate S1:

security rule fixed

EC2 → RDS path preserved

Identity:

relevant rule = MODIFIED

Impact:

required invariants = AFFECTED

Verification:

SEC = PASS

FUNC = PASS

Promotion:

eligible for PROMOTE

Impact ensures the relevant obligations are checked; it never authorizes promotion itself.

30. Adversarial Cases

Case	Required behavior
Unrelated tag change	Do not invalidate unless tag is explicitly in invariant scope.
SG rule change	Invalidate relevant SG-based obligations.
Subnet association change	Consider network/function invariants through supported paths.
Created resource	Generate applicable baseline checks.
Deleted dependency	Affected dependent obligations require verification.
Unknown identity	Do not preserve relevant old evidence.
Unknown dependency edge	Do not conclude unaffected.
Graph cycle	Terminate safely with visited set.
Multiple paths to same invariant	Deduplicate result; preserve all reasons.

31. Deduplication

```
affected[invariant_id] = merge(  
    status,
```

```
    impact_types,  
    changed_resources,  
    dependency_paths,  
    evidence_invalidations  
)
```

One logical invariant record may contain multiple independent impact reasons and paths.

32. Determinism

- Sort changes by stable semantic ID/address.
- Traverse adjacency deterministically.
- Sort dependency paths deterministically.
- Deduplicate before serialization.
- Include algorithm/graph/invariant versions in output.
- Same compatible inputs must yield the same impact semantics and hash.

33. Complexity

For the MVP, indexed graph traversal should be approximately $O(V+E)$ over the reachable subgraph per traversal set. Shared traversal/caching is allowed provided cache keys include all semantic versions required for correctness.

34. Caching

- Cache only immutable inputs/results.
- Cache keys include state hash, candidate hash, graph version, invariant registry version, and impact algorithm version where relevant.
- Never reuse across incompatible normalization/invariant versions.
- Caching must not hide changed uncertainty or evidence compatibility.

35. Unit Tests

- Direct scope impact.
- Dependency-path impact.
- Reverse traversal.
- Created-resource baseline.
- Deletion.
- Replacement.
- Identity uncertainty.
- Dependency uncertainty.
- Multiple-path deduplication.
- No-edge semantics.
- Cycle termination.
- Deterministic output.

36. Integration Tests

- IdentityReport → ImpactReport.
- DependencyGraph → impact closure.
- Trusted evidence → invalidation.
- ImpactReport → Verification Work Set.
- Rejected candidate leaves trusted evidence intact.
- Safe candidate triggers required checks before promotion.

37. Acceptance Criteria

- Every meaningful change type is considered as an impact source.
- Directly scoped protected invariants are identified.
- Supported dependency paths can invalidate evidence.
- Reverse impact traversal works where necessary.
- Identity/dependency uncertainty is conservatively propagated.
- Historical evidence is retained while new-state eligibility is invalidated.
- Created-resource baseline verification is generated where applicable.
- Multiple impact paths are deduplicated with explanations.
- Impact results are deterministic and versioned.
- Impact never claims an invariant is violated.

38. Non-Goals

- Proving an invariant.
- Choosing PROMOTE/REJECT.
- Generating repair patches.
- Using LLM reasoning for impact decisions.
- Building an unrestricted AWS dependency graph.
- Assuming graph completeness.
- Deleting or rewriting historical evidence.

39. Component Boundary

Component	Responsibility
Identity Engine	Determine correspondence/change facts.
Dependency Engine	Build supported graph and relationship facts.
Impact Engine	Determine affected obligations and verification work.
Verification Engine	Determine current property truth.
Promotion Controller	Determine whether candidate may become trusted.

40. Final Impact/Invalidation Directive

DAIL shall conservatively invalidate protected evidence whenever a supported direct or dependency relationship makes prior proof potentially stale. It shall never confuse AFFECTED with VIOLATED and shall never use absence of a known dependency as proof that no dependency exists.

The output is a justified verification work set. Evidence may be carried forward only when its validity remains explicitly established under the state, identity, dependency, invariant-version, and verifier-compatibility rules.

Appendix A — Core Algorithm

```
def compute_impact(trusted, candidate, identity, graph, invariants):
    changes = identity.changed_resources()
    affected = {}

    for change in changes:
        for inv in invariants:
            if direct_scope_match(inv, change):
                affected[inv.id].add(DIRECT, change)

        closure = traverse_impact_graph(change, graph)

        for node, path in closure:
            for inv in invariants_scoped_by(node):
                affected[inv.id].add(DEPENDENCY, change, path)

    propagate_uncertainty(change, graph, affected)

    for inv in affected:
        invalidate_prior_evidence(inv, trusted)
        create_verification_work(inv)

    add_new_resource_baseline_work(changes)
    deduplicate_and_sort()
    return ImpactReport(...)
```

Appendix B — Next Specification

Document 09 — Verification & Promotion Specification consumes the verification work set produced here and defines exact verification predicates, evidence sufficiency rules, the promotion decision matrix, blocking conditions, and the promotion transaction policy.